

# Danh Sách Vòng Đơn (Singly Circular Linked List)

Khái niệm, Cấu trúc và Ứng dụng với C++

TS Phạm Hữu Tùng

Nguồn hình ảnh: 21st Century C++ - Communications of the ACM

# Khái niệm và Đặc điểm của Danh sách vòng đơn

## Định nghĩa Danh sách vòng đơn

Danh sách liên kết vòng (Circular Linked List) là một cấu trúc dữ liệu trong đó các phần tử được liên kết với nhau theo một vòng tròn.

Trong trường hợp **Danh sách vòng đơn (Singly Circular Linked List)**, đây là một biến thể của danh sách liên kết đơn mà trong đó con trỏ `next` của nút cuối cùng (tail) không trở về `NULL` mà trở ngược về nút đầu tiên (head) của danh sách, tạo thành một chu trình liên tục.

## So sánh với Danh sách liên kết đơn

### Danh sách liên kết đơn

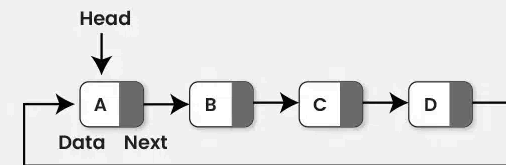
- Nút cuối trở tới `NULL`.
- Duyệt có điểm dừng rõ ràng.
- Truy cập nút cuối yêu cầu duyệt `O(N)` (nếu không có con trỏ tail).

### Danh sách vòng đơn

- Nút cuối trở về `head`.
- Duyệt liên tục, cần điều kiện dừng.
- Truy cập `head`/`tail` mất `O(1)` (nếu có con trỏ `last`).

## Minh họa Cấu trúc

### Circular Linked List



Minh họa cấu trúc cơ bản của Danh sách vòng đơn, nơi nút cuối cùng liên kết trở lại nút đầu tiên.

## Ưu điểm và Nhược điểm khi sử dụng

### ✓ Ưu điểm

- Duyệt liên tục:** Dễ dàng duyệt toàn bộ danh sách từ bất kỳ nút nào và quay lại điểm xuất phát.
- Truy cập hiệu quả:** Nếu duy trì con trỏ đến nút cuối cùng, việc truy cập nút đầu và cuối chỉ mất `O(1)`.
- Ứng dụng đa dạng:** Thích hợp cho các cấu trúc vòng như bộ đệm, hàng đợi vòng, hoặc lập lịch quay vòng.

### ✗ Nhược điểm

- Thao tác phức tạp hơn:** Thêm/xóa nút đòi hỏi xử lý cẩn thận các trường hợp cạnh để duy trì tính vòng.
- Nguy cơ vòng lặp vô hạn:** Cần có điều kiện dừng rõ ràng trong quá trình duyệt để tránh lặp vô hạn.
- Tìm kiếm nút:** Có thể phức tạp hơn một chút do không có `NULL` làm điểm dừng tự nhiên.

# Cấu trúc dữ liệu trong C++

## 1. Biểu diễn Nút (Node)

- Trong danh sách liên kết, **Node** là đơn vị cơ bản, mỗi node chứa hai thành phần chính: **dữ liệu (data)** và một **con trỏ (next)**.
- Phần **data** lưu trữ thông tin thực tế (ví dụ: số nguyên, chuỗi ký tự, đối tượng).
- Phần **next** là một con trỏ kiểu **Node\*** dùng để lưu địa chỉ của nút kế tiếp trong danh sách, tạo thành mối liên kết.

## 2. Cấu trúc Danh sách vòng đơn

- Một **Danh sách vòng đơn** được quản lý bởi một cấu trúc hoặc lớp riêng biệt.
- Thay vì quản lý con trỏ `head`, cách hiệu quả hơn là chỉ cần quản lý con trỏ **last** (nút cuối cùng).
- Từ nút **last**, ta có thể dễ dàng truy cập nút đầu tiên (head) thông qua `last->next` với độ phức tạp  $O(1)$ .

## 3. Quản lý Con trỏ 'head'/'last'

Với DSLK Vòng, việc duy trì chỉ một con trỏ **last** mang lại nhiều lợi ích, giúp tối ưu hóa các thao tác quan trọng:

→ **Truy cập Head:**  $O(1)$  qua `last->next`

→ **Truy cập Tail:**  $O(1)$  qua `last`

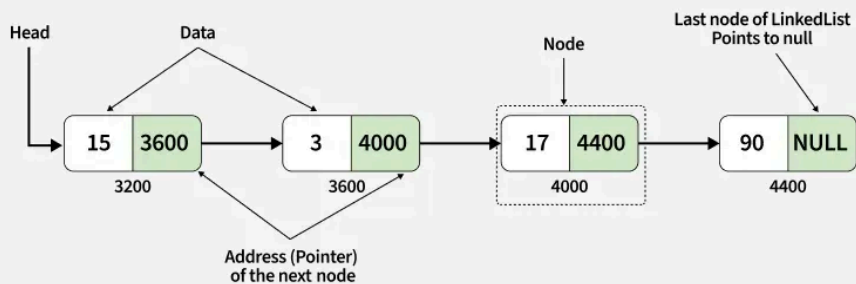
→ **Thêm vào cuối:**  $O(1)$

→ **Dễ quản lý:** Đơn giản hóa logic vòng

### Cấu trúc Node trong C++

```
// Biểu diễn một nút trong DSLK
struct Node {
    int data;      // Dữ liệu của nút
    Node* next;    // Con trỏ tới nút kế tiếp

    // Constructor để khởi tạo nút
    Node(int val) : data(val), next(nullptr) {}
};
```



Minh họa cấu trúc một nút (Node)

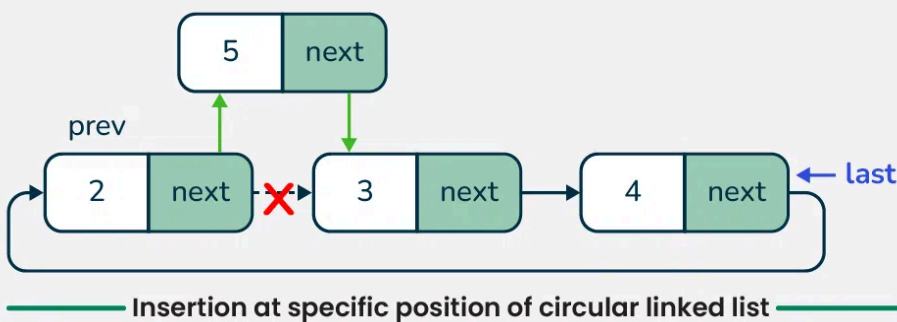
### Lớp SinglyCircularLinkedList

```
// Lớp quản lý Danh sách vòng đơn
class SinglyCircularLinkedList {
private:
    Node* last; // Con trỏ tới nút cuối cùng

public:
    // Constructor khởi tạo danh sách rỗng
    SinglyCircularLinkedList() : last(nullptr) {}

    // Phương thức để lấy nút đầu tiên (head)
    Node* getHead() const {
        if (last == nullptr) return nullptr;
        return last->next;
    }

    // ... các phương thức khác ...
};
```



# Các thao tác cơ bản: Thêm nút & Mã nguồn C++

## 1. Khởi tạo danh sách rỗng

```
// Constructor của lớp SinglyCircularLinkedList
SinglyCircularLinkedList() : head(nullptr), last(nullptr) {}
```

- ▶ Khi một đối tượng `SinglyCircularLinkedList` mới được tạo, cả `head` và `last` đều được gán `nullptr`.
- ▶ Điều này chỉ ra rằng danh sách hiện tại đang ở trạng thái rỗng, chưa chứa bất kỳ nút nào.
- ▶ Khởi tạo đúng là bước quan trọng đầu tiên trước khi thực hiện các thao tác khác.

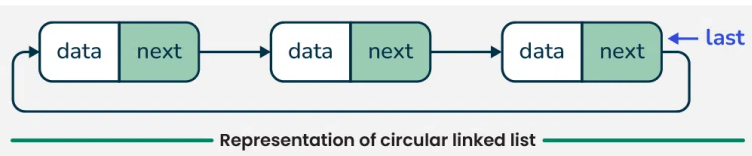
## 2. Thêm nút vào đầu danh sách (addHead)

```
void addHead(int val) {
    Node* newNode = new Node(val); // 1. Tạo nút mới
    if (head == nullptr) { // 2. DS rỗng
        head = newNode;
        last = newNode;
        newNode->next = head; // Trỏ về chính nó
    } else { // 3. DS không rỗng
        newNode->next = head; // Nút mới trỏ tới head cũ
        head = newNode;      // Cập nhật head
        last->next = head;   // last->next trỏ tới head mới
    }
}
```

### Logic hoạt động:

- **DS rỗng:** Nút mới là `head` & `last`. Con trỏ `next` của nó trỏ về chính nó.
- **DS không rỗng:** Nút mới trỏ tới `head` cũ, `head` được cập nhật. `last->next` phải trỏ tới `head` mới để duy trì vòng.

head last newNode O(1) thêm đầu



## 3. Thêm nút vào cuối danh sách (addTail)

```
void addTail(int val) {
    Node* newNode = new Node(val); // 1. Tạo nút mới
    if (head == nullptr) { // 2. DS rỗng
        head = newNode;
        last = newNode;
        newNode->next = head;
    } else { // 3. DS không rỗng
        newNode->next = head; // Nút mới trỏ tới head
        last->next = newNode; // last cũ trỏ tới nút mới
        last = newNode;      // Cập nhật last
    }
}
```

### Logic hoạt động:

- **DS rỗng:** Tương tự như `addHead`.
- **DS không rỗng:** `last` cũ trỏ tới nút mới. Sau đó, `last` được cập nhật thành nút mới, và `next` của `last` mới này trỏ về `head`.

head last newNode O(1) thêm cuối



## Ví dụ Mã nguồn minh họa: Hàm hiển thị Danh sách

```
void display() const {
    if (head == nullptr) { return; }
    Node* current = head;
    do {
        std::cout << current->data << " -> ";
        current = current->next;
    } while (current != head);
    std::cout << "(head)";
}
```

Hàm `display()` minh họa cách duyệt qua danh sách vòng. Quá trình bắt đầu từ `head` và tiếp tục cho đến khi con trỏ `current` quay trở lại `head`, đảm bảo tất cả các nút được hiển thị đúng một lần mà không bị lặp vô hạn.

# Các thao tác cơ bản: Xóa nút & Mã nguồn C++

## 1. Kiểm tra danh sách rỗng trước khi xóa

```
bool isEmpty() const {  
    return head == nullptr;  
}  
  
// ... trong các hàm xóa:  
if (isEmpty()) {  
    std::cout << "Danh sách rỗng...\n";  
    return;  
}
```

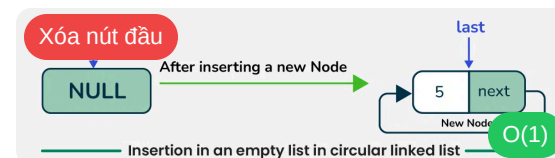
Trước khi thực hiện bất kỳ thao tác xóa nào, việc kiểm tra xem danh sách có rỗng hay không là bắt buộc để tránh các lỗi truy cập con trỏ `nullptr` và đảm bảo tính ổn định của chương trình. Nếu danh sách rỗng, thao tác xóa sẽ không được thực hiện.

## 2. Xóa nút ở đầu danh sách (deleteHead)

```
void deleteHead() {  
    if (head == nullptr) return;  
  
    Node* nodeToDelete = head;  
    // Trường hợp chỉ có một nút  
    if (head == last) {  
        head = nullptr;  
        last = nullptr;  
    } else {  
        // Trường hợp có nhiều nút  
        head = head->next;  
        last->next = head;  
    }  
    delete nodeToDelete;  
}
```

### Logic hoạt động:

- **1 nút:** Nếu `head == last`, danh sách trở nên rỗng, `head` và `last` gán bằng `nullptr`.
- **>1 nút:** `head` được chuyển sang nút kế tiếp. `last->next` phải được cập nhật để trở tới `head` mới, duy trì tính vòng.
- **Giải phóng:** Luôn dùng `delete` để tránh rò rỉ bộ nhớ.

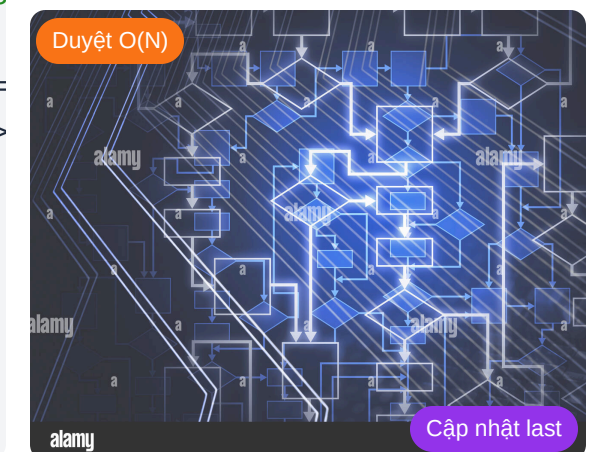


## 3. Xóa nút ở cuối danh sách (deleteTail)

```
void deleteTail() {  
    if (head == nullptr) return;  
  
    Node* nodeToDelete = last;  
    // Trường hợp chỉ có một nút  
    if (head == last) {  
        head = nullptr;  
        last = nullptr;  
    } else {  
        // Phải duyệt tìm nút trước  
        Node* current = head;  
        while (current->next !=  
            current) {  
            current = current->  
                next;  
        }  
        current->next = head;  
        last = current;  
    }  
    delete nodeToDelete;  
}
```

### Logic hoạt động:

- **1 nút:** Tương tự `deleteHead`, danh sách trở nên rỗng.
- **>1 nút:** Cần duyệt từ `head` để tìm nút ngay trước `last`. Sau đó, cập nhật `current->next` trở về `head` và gán lại con trỏ `last`.
- **Độ phức tạp:** Thao tác này có độ phức tạp  $O(N)$ .



### Hiệu suất & Logic



`deleteHead` có hiệu suất  $O(1)$  nhờ cập nhật con trỏ trực tiếp. Ngược lại, `deleteTail` yêu cầu duyệt ( $O(N)$ ) để tìm nút trước đó.

$O(1)$  vs  $O(N)$

Duyệt

Con trỏ

### Quản lý Bộ nhớ & An toàn



Sử dụng `delete` để trả lại bộ nhớ, ngăn ngừa rò rỉ. Xử lý cẩn thận trường hợp danh sách chỉ có một nút là rất quan trọng.

Giải phóng

Tránh rò rỉ

Trường hợp đặc biệt

# Các thao tác nâng cao & Mã nguồn C++

## 🕒 1. Duyệt và In các phần tử của danh sách

```
void display() {
    // 1. Kiểm tra danh sách có rỗng không
    if (head == nullptr) { return; }

    Node* current = head;
    // 2. Dùng do-while để xử lý head trước
    do {
        std::cout << current->data << " -> ";
        current = current->next;
    } while (current != head); // 3. Dừng khi quay lại head
    std::cout << "(head)";
}
```

Logic duyệt danh sách vòng đơn phải có điều kiện dừng rõ ràng để tránh lặp vô hạn. Vòng lặp **do-while** là lựa chọn lý tưởng vì nó đảm bảo nút `head` được xử lý ít nhất một lần, ngay cả khi danh sách chỉ có một phần tử.

- **Kiểm tra rỗng:** Luôn là bước đầu tiên để đảm bảo an toàn.
- **Bắt đầu từ `head`:** Khởi tạo con trỏ `current` tại `head`.
- **Điều kiện dừng:** Vòng lặp kết thúc khi `current` quay trở lại `head`.

Duyệt vòng

do-while

Kiểm tra rỗng

Điều kiện dừng

## 🔍 2. Tìm kiếm một phần tử trong danh sách

Thao tác tìm kiếm duyệt qua danh sách để tìm nút đầu tiên có giá trị khớp với khóa (key). Nếu tìm thấy, trả về con trỏ tới nút đó; nếu không, trả về `nullptr` sau khi đã duyệt hết một vòng.

- **Trường hợp không tìm thấy:** Khi duyệt hết vòng mà không khớp giá trị nào, hàm trả về `nullptr`.
- **Hiệu suất:** Trong trường hợp xấu nhất (phần tử ở cuối hoặc không có), độ phức tạp là O(N).

Tìm thấy

Không tìm thấy (nullptr)

O(N)

So sánh giá trị

```
Node* search(int key) {
    if (head == nullptr) { return nullptr; }

    Node* current = head;
    do {
        if (current->data == key) {
            return current; // Tìm thấy
        }
        current = current->next;
    } while (current != head);

    return nullptr; // Không tìm thấy
}
```

## 🗑️ 3. Xóa một nút bất kỳ theo giá trị

```
void deleteByValue(int key) {
    if (head == nullptr) return;
    Node* current = head, *prev = nullptr;

    // Tìm nút cần xóa và nút trước nó
    do {
        if(current->data == key) break;
        prev = current;
        current = current->next;
    } while(current != head);

    // Nếu không tìm thấy nút
    if (current->data != key) return;

    // Nếu chỉ có 1 nút
    if (current == head && current->next == head) {
        head = last = nullptr; delete current; return;
    }

    // Xóa head
    if (current == head) {
        prev = last; head = head->next;
    }
    // Xóa last
    else if (current == last) { last = prev; }

    prev->next = current->next;
    delete current;
}
```

Đây là thao tác phức tạp nhất, đòi hỏi phải xử lý nhiều trường hợp đặc biệt (cạnh) để tránh lỗi và đảm bảo tính toàn vẹn của danh sách.

- **Dùng 2 con trỏ:** `current` để chỉ nút đang xét và `prev` để chỉ nút trước đó.
- **Xóa `head`:** Cần cập nhật `head` mới và `last->next`.
- **Xóa `last`:** Cần cập nhật con trỏ `last` về nút `prev`.
- **DS một nút:** Danh sách trở nên rỗng sau khi xóa.
- **Giải phóng bộ nhớ:** Luôn dùng `delete` để tránh rò rỉ bộ nhớ.

Trường hợp cạnh

Giải phóng bộ nhớ

prev & current

Cập nhật last

# Ứng dụng và Kết luận

## 🔧 1. Ứng dụng thực tế

☰ **Bộ đệm vòng (Circular Buffer):** Quản lý dữ liệu trong các hệ thống truyền thông (audio, video) hoặc I/O, nơi dữ liệu mới ghi đè lên dữ liệu cũ nhất khi đầy.

Hàng đợi

FIFO

Tái sử dụng

🕒 **Đa nhiệm (Task Scheduling):** Trong HĐH, quản lý tác vụ theo cơ chế Round-Robin, mỗi tác vụ được cấp một khoảng thời gian xử lý.

Round-Robin

OS

Công bằng

⚙️ **Quản lý bộ nhớ:** Theo dõi các khối bộ nhớ khả dụng, cho phép duyệt liên tục để tìm kiếm hoặc thu hồi.

📺 **Trình chiếu lặp lại:** Ứng dụng trong các trình chiếu nơi hình ảnh hoặc video cần lặp lại liên tục.

## ↔️ 2. So sánh và Lựa chọn

### Ưu điểm nổi bật

✅ Dễ dàng duyệt và lặp lại tuần hoàn, phù hợp với các tác vụ truy cập liên tục.

⚡ Thao tác `addTail` và `deleteHead` hiệu quả  $O(1)$  khi có con trỏ `last`.

### Hạn chế cần cân nhắc

⚠️ Thao tác `deleteTail` và tìm kiếm có độ phức tạp  $O(N)$ .

🕒 Quản lý con trỏ phức tạp, cần cẩn trọng để tránh lặp vô hạn.

### Quyết định lựa chọn

💡 Chọn khi việc duyệt tuần hoàn là trọng tâm, hoặc khi thao tác ở hai đầu là phổ biến.

↔️ Với tìm kiếm/xóa linh hoạt, cân nhắc B-tree, Hash Table, DSLK đôi.

## 📄 3. Kiến thức trọng tâm



### Cấu trúc

Nút cuối trở về nút đầu, tạo thành một vòng khép kín.



### Hiệu suất

Thêm/xóa đầu, thêm cuối  $O(1)$ . Xóa cuối, tìm kiếm  $O(N)$ .



### Quản lý

Cẩn trọng với trường hợp rỗng/1 nút và điều kiện dừng khi duyệt.

## 4. Hỏi & Đáp



❓ Mở rộng thảo luận về ứng dụng, so sánh và các vấn đề liên quan.